



Universidad Técnica Estatal de Quevedo
Facultad de Ciencias de la Computación
Carrera de Ingeniería de Software

Título

Consumo de Web Services REST con Angular 22

Nombre:

Irvin Marcelo Cajas Ibarra

Asignatura

Aplicaciones Web

Semestre

5to Software "A"

Fecha:

10/7/2026

1. OBJETIVO DE LA PRÁCTICA

Implementar el consumo de una API REST de forma asíncrona y reactiva utilizando las características modernas de Angular 22 (arquitectura Zoneless, Signals e httpResource), diagnosticando el error de intercambio de recursos de origen cruzado (CORS) y resolviéndolo mediante la configuración de un proxy de desarrollo inverso.

2. ARCHIVOS DE CONFIGURACIÓN Y CÓDIGO FUENTE

2.1. Inyección de HttpClient (src/app/app.config.ts)

```
import { ApplicationConfig } from '@angular/core';
import { provideHttpClient } from '@angular/common/http';

// La configuración de la aplicación es un objeto de proveedores
(providers).
export const appConfig: ApplicationConfig = {
  providers: [
    // Registra HttpClient para poder inyectarlo en servicios y
    componentes.
    // En Angular 22 no hace falta withFetch(): Fetch ya es el backend
    por defecto.
    provideHttpClient(),
  ],
};
```

2.2. Configuración de Redirección del Proxy (proxy.conf.json)

```
{
  "/api": {
    "target": "http://localhost:8080",
    "secure": false,
    "changeOrigin": true,
    "pathRewrite": {
      "^/api": ""
    }
  }
}
```

2.3. Servicio CRUD con Manejo de Errores (src/app/producto.service.ts)

```
import { Injectable, inject } from '@angular/core';
import { HttpClient, HttpResponseError } from '@angular/common/http';
import { Observable, catchError, throwError } from 'rxjs';
import { Producto } from '../producto.model';

// @Injectable con providedIn: 'root' crea UNA sola instancia para toda
la app.
@Injectable({ providedIn: 'root' })
```

```

export class ProductoService {
  // URL base del recurso. Es relativa para que funcione con el proxy
  (paso 7).
  private readonly base = '/api/productos';

  // inject() obtiene HttpClient sin necesitar un constructor.
  private readonly http = inject(HttpClient);

  // GET colección: devuelve un Observable con un arreglo de productos.
  // El <Producto[]> le dice a Angular el tipo esperado de la respuesta
  JSON.
  listar(): Observable<Producto[]> {
    return this.http
      .get<Producto[]>(this.base)
      .pipe(catchError((err) => this.manejarError(err)));
  }

  // GET por id: obtiene un único recurso.
  obtener(id: number): Observable<Producto> {
    return this.http
      .get<Producto>(`${this.base}/${id}`)
      .pipe(catchError((err) => this.manejarError(err)));
  }

  // POST: crea un recurso. El segundo argumento es el cuerpo (body)
  enviado.
  // Angular serializa el objeto a JSON automáticamente.
  crear(p: Omit<Producto, 'id'>): Observable<Producto> {
    return this.http
      .post<Producto>(this.base, p)
      .pipe(catchError((err) => this.manejarError(err)));
  }

  // PUT: reemplaza por completo el recurso identificado por id.
  actualizar(id: number, p: Producto): Observable<Producto> {
    return this.http
      .put<Producto>(`${this.base}/${id}`, p)
      .pipe(catchError((err) => this.manejarError(err)));
  }

  // DELETE: elimina el recurso. Suele responder 204 No Content (sin
  cuerpo).
  eliminar(id: number): Observable<void> {
    return this.http
      .delete<void>(`${this.base}/${id}`)
      .pipe(catchError((err) => this.manejarError(err)));
  }

  // Manejo centralizado de errores por familia de código de estado
  (Paso 6).
  private manejarError(err: HttpErrorResponse) {
    if (err.status === 0) {
      console.error('Sin red o bloqueo CORS (ver paso 7:
proxy.conf.json)');
    } else if (err.status === 404) {
      console.error('Recurso no encontrado (404)');
    } else if (err.status >= 500) {

```

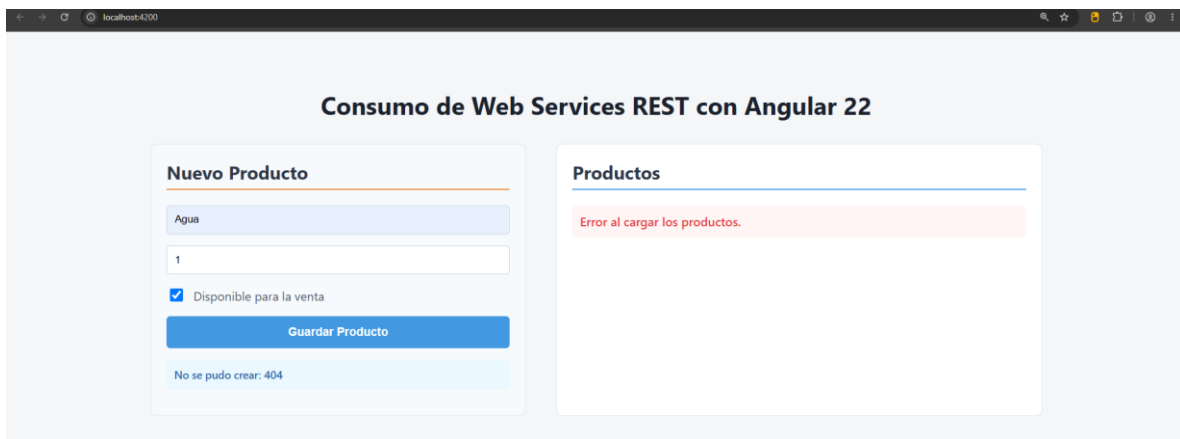
```

    console.error('Error del servidor (5xx)');
  } else if (err.status >= 400) {
    console.error(`Error del cliente (${err.status})`);
  }
  // Re-emite el error para que quien se suscriba pueda reaccionar.
  return throwError(() => err);
}
}

```

3. CAPTURAS DE EJECUCIÓN

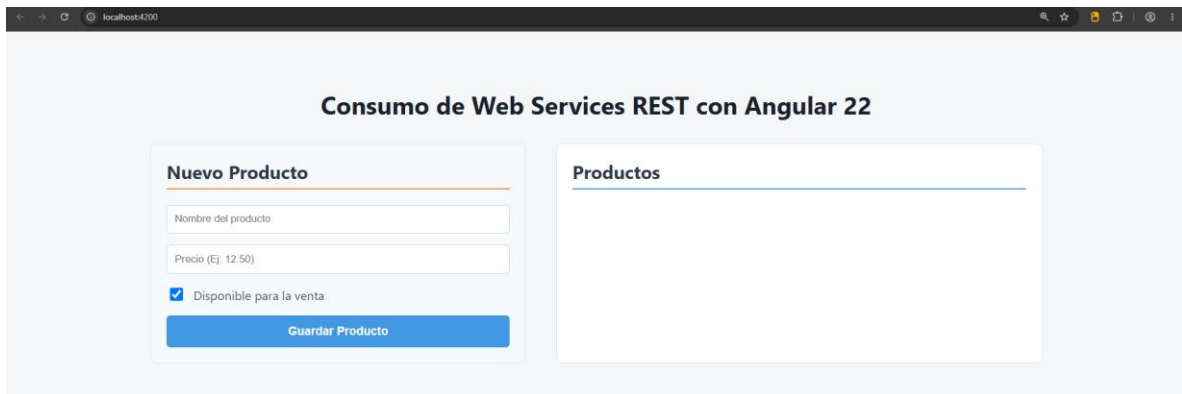
3.1. Captura del Error de CORS Detectado (sin el proxy conectado)



3.2. Servidor de Datos Local Activo (json-server)



3.3. Interfaz Operativa con Carga de Datos Exitosa



3.4. Confirmación Visual de Operación Reactiva (POST)

